

CampusHub 校园论坛 · 中期报告

姓名：杨坤毅（学号 24320314）

角色：团队 PM + 评论模块全栈负责人

时间：2026.9（D1 – D4）

仓库：<https://github.com/YangKunYi24320314/postingweb>

首先，我带领团队进行立项，由曾繁一同学提出的项目得到全体的认可。经过全体讨论，我们决定制作面向校园的实名发帖社区论坛。解决其他相关产品自由度和缺乏结构化的互动问题。核心功能包括：用户认证 / 帖子发布 / 评论互动 / 点赞收藏 / 记录中心 / 推荐排序。

作为项目 PM，我负责制定技术规范和书写契约文档。经过考量，我决定让团队采取目前主流的开发模式和开发技术。

- 技术栈：Vue 3 + Vite（前端）· Node.js + Express（后端）· PostgreSQL（数据库）
- 协作：Apifox 定接口 · Git + GitHub 并行开发 · PR 代码审查

团队分工（全栈开发，每人独立 Git 库）

成员	负责模块
杨坤毅	PM · 评论（列表/发表/编辑/删除）
王涵宇	认证（注册/登录/个人信息/分类标签）
邵家晨	帖子（发布/编辑/删除/列表/详情）
杨尔灏	互动（点赞/取消/收藏/取消收藏）
曾繁一	记录中心（浏览记录/我的帖子/收藏/点赞）

我们的并行开发计划：[postingweb/devdocs/development-plan.md](https://github.com/YangKunYi24320314/postingweb/blob/main/devdocs/development-plan.md) at main · YangKunYi24320314/postingweb

之后三天，我的任务主要是三条主线

① 项目管理（PM）

- GitHub 以及 Gitee 创建与管理、分支策略、协作规范
- 制定接口/数据库契约，审查代码、合并 PR
- 修改冲突和组员不合规代码

② 评论模块全栈

- 后端 4 接口 + 前端评论区（楼中楼）

③ 工程基建

- 统一请求封装、统一响应、公共鉴权、模块化骨架、局域网部署

工作量速览（来自 Git）

- 个人提交：77 次
- PR 合并：25 次（#1 – #33）
- 开发分支：feature/comment · feature/commend · feature/LAN
- 核心交付文件：

server/routes/comments.js（203 行，4 接口）

client/src/api/request.js（65 行，全队封装）

client/src/views/PostDetailView.vue（430 行）（主要为合规性修改）

client/src/components/CommentItem.vue（266 行）

server/serve-lan.ps1 + devdocs/lan-deploy.md（局域网方案）

以及页面框架，前后端基本文件结构等等项目骨架的创建也由我完成

具体的内容

评论后端 + 工程骨架

- 后端评论路由 server/routes/comments.js：列表/发表/编辑/删除
- 统一 ok/fail 返回、\$1 \$2 参数化查询防注入
- 评论数增减与关系表写入放同一事务
- 本地建库 campushub + 10 张表 + 种子数据，真实库跑通（含 401/403/404/400 分支）
- 前端统一封装 client/src/api/request.js + 示例 api/auth.js
- Vite 代理 /api → 127.0.0.1:3000，前端不管跨域

- server.js 改为自动加载 routes/ 下所有文件挂到 /api
- 把 optionalAuth 提取到公共 middleware/auth.js, 全队共用

评论区前端 + 代码审查

- 新增 PostDetailView.vue: 帖子全文 + 楼中楼评论树 + 点赞/收藏 + 上报浏览 + 附件下载 + 删帖
- 新增 CommentItem.vue: 单条评论 (点赞/回复/编辑/删除), 支持无限嵌套
- 新增 api/comments.js: 列表/发表/编辑/删除, 全走 request.js
- 路由新增 /post/:id 详情页; 样式全用 tokens.css 变量; 未登录弹提示
- 审查并合并 feature/commend(#20)、feature/post-detail(#21/#22)、feature/recommendation(#24)
- 清理旧版违规实现: server/src/, utils/request.js 重复封装、无用素材

局域网联机 + 契约统一

- 局域网部署方案 B (单端口发布): server.js 托管 client/dist + SPA 回退 + 未匹配 /api 返回 {code:1004}
- PORT 从 .env 读取 (默认 3000); 旧 /upload 改用 req.get('host') 拼地址
- 新增 server/serve-lan.ps1 一键脚本: 排除 WSL/虚拟网卡打印局域网 IPv4、放行防火墙 TCP 3000、启动后端
- 新增 devdocs/lan-deploy.md 完整联机指南
- 契约统一: avatar_url → avatarUrl (驼峰); 移除未实现 /recommend/posts, 推荐并入 ?rank=recommend; 修正 rank YAML 语法
- 合并 #27 records-detail、#28 develop、#30 LAN、#33 recommendation-decay

详细的实现方式

接口	说明
GET /api/posts/:id/comments	公开, 返回评论列表 (按时间正序), 登录后附 isLiked

POST /api/posts/:id/comments	需登录，支持 parentId 楼中楼，评论数 +1 在事务内
PUT /api/comments/:id	需登录，仅作者（非作者 403），改内容
DELETE /api/comments/:id	需登录，仅作者或管理员，软删除，评论数-1

① 楼中楼校验

- parentId 必须存在，且必须属于同一个帖子，防止跨帖子挂评论

② 事务一致性

- 发表/删除评论：同时写 comments 表 + 更新 posts.comment_count
- 一个事务里 BEGIN→操作→COMMIT，失败 ROLLBACK，避免脏数据

③ 权限控制

- 编辑仅限作者；删除仅限作者或管理员（查 users.role 判断）

④ 计数安全

- 删除评论用 GREATEST(comment_count - 1, 0)，防止减成负数

评论模块前端实现

- 树组装：把扁平评论列表按 parentId 拼成树 (commentTree computed)
- 交互：发评论、回帖、点赞、编辑、删除，变更后统一 handleReload 刷新
- 状态：未登录弹「登录后才能评论」；显示我的 userId 判断「是不是我的评论」
- 附件：详情页支持下载 (handleDownload → /api/attachments/:id/download)
- 删除帖子：仅作者或管理员，软删除后回到帖子广场

前端统一请求封装 request.js

- 约定：页面禁止直接写 axios/fetch，只走 src/api/request.js

这样做在协作开发中有很多好处：

自动带 token → 请求头 Authorization: Bearer <token>

自动解包 → 后端 { code, message, data }，成功直接返回 data

集中错误 → code=1002 清 token 跳登录页，其余统一提示

- 导出 saveToken / getToken / clearToken 管理 localStorage 的 token
- 每个功能模块一个文件: auth.js / comments.js / post.js / record.js ...

后端公共基建

- server/utils/response.js: ok / fail + 错误码 (1001/1002/1003/1004/1005/5000), 三层返回
- server/middleware/auth.js: auth (必须登录) + optionalAuth (可选登录), 全队共用
- server/db.js: pg 连接池统一配置, 参数从 .env 读取, 缺省兜底
- server.js: 自动加载 routes/ 下所有模块挂 /api, 新增模块只需加一个文件, 不用改公共入口
- server/utils/auth-validation.js: 注册/登录入参校验统一

契约文档管控

- 维护 devdocs/: api-protocol.md · openapi.yaml · database-schema.md · development-plan.md
- 逐一核对队友接口与契约是否一致 (返回格式/字段名/鉴权/计数/是否走统一封装/只用 tokens.css 变量)
- 新增: database-schema.md 补 post_attachments 表; api-protocol.md / openapi.yaml 新增附件上传/下载
- 统一字段: avatar_url → avatarUrl (驼峰), 同步组件
- 移除未实现接口: /recommend/posts → 并入 GET /api/posts?rank=recommend, 修正 rank YAML
- users.background_url 类型改 TEXT, 标注个人主页背景图

工程规范

- 样式只用 src/styles/tokens.css 变量 (--brand-primary / --space-md ...), 不写死色值
- 页面放 src/views/, 组件放 src/components/, 公共布局在 BaseLayout.vue

- 调后端必须走 src/api/request.js 封装
- 统一用 Element Plus (main.js 已全局注册), 模板直接写 <el-xxx>
- ESLint + Prettier + .editorconfig, 写完跑 npm run format / lint

第一次试着带领团队协作开发, 过程中不免遇到一些经典的问题:

① 评论数与关系表不一致

→ 评论写入/删除与计数更新放同一事务, 失败 ROLLBACK; 删除用 GREATEST 防负数

② POST 路由冲突 (/post/:id)

→ 我的 PostDetailView.vue 与「帖子组」的 PostDetail.vue 同地址同路由名, 合并后评论区"消失"

→ 删重复路由, 保留 PostDetailView 为唯一详情页, 并入附件下载+删帖

③ 公共入口被整文件覆盖

→ 合并后 server.js 被旧版整文件替换, 模块没挂载、前端不代理 /api

→ 以 devdocs/* 契约为准重建归一化, 删违规残余

④ 代码审查工作量大、写法不统一

→ 用 AI 协作工具逐文件对照 api-protocol.md 核点, 集中反馈并提醒"只加路由文件/别动公共入口"

⑤ 旧 /upload 写死 127.0.0.1

→ 改用 req.get('host') 相对拼地址, 局域网/部署下图片正常

⑥ 端口写死 + 日志打印数据库密码

→ PORT 从 .env 读 (默认 3000); 删除打印密码的调试日志

目前我们取得了阶段性的成果

- 评论模块闭环: 详情页看正文 / 发评论 / 楼中楼 / 点赞 / 收藏 / 记录浏览
- 全队统一: request.js 封装 + 统一响应 + 公共鉴权 + 模块化骨架

- 契约与实现一致：附件、驼峰字段、推荐并入 rank 全部同步
- 演示能力：任意组员电脑「一键当服务器」，他人浏览器直接访问
- 团队协作：25 次 PR 合并全部审查通过，代码规范统一

后续计划

- 继续推进模块开发，推动局域网联机演示
- 继续审查/合并队友代码，核对与 api-protocol.md 一致性，处理合并遗留项
- 增加功能和美化前端 UI
- 尚有一些前端的 BUG 等待修改

以下附上主要页面的浏览器截图